Paths

Challenges of this unit

* The "secret" of all "easy" optimization problems
* Some applications of shortest paths
* Two ways of formulating the shortest path problem
* Some shortest path algorithms
* How to select the best one

Links with previous knowledge

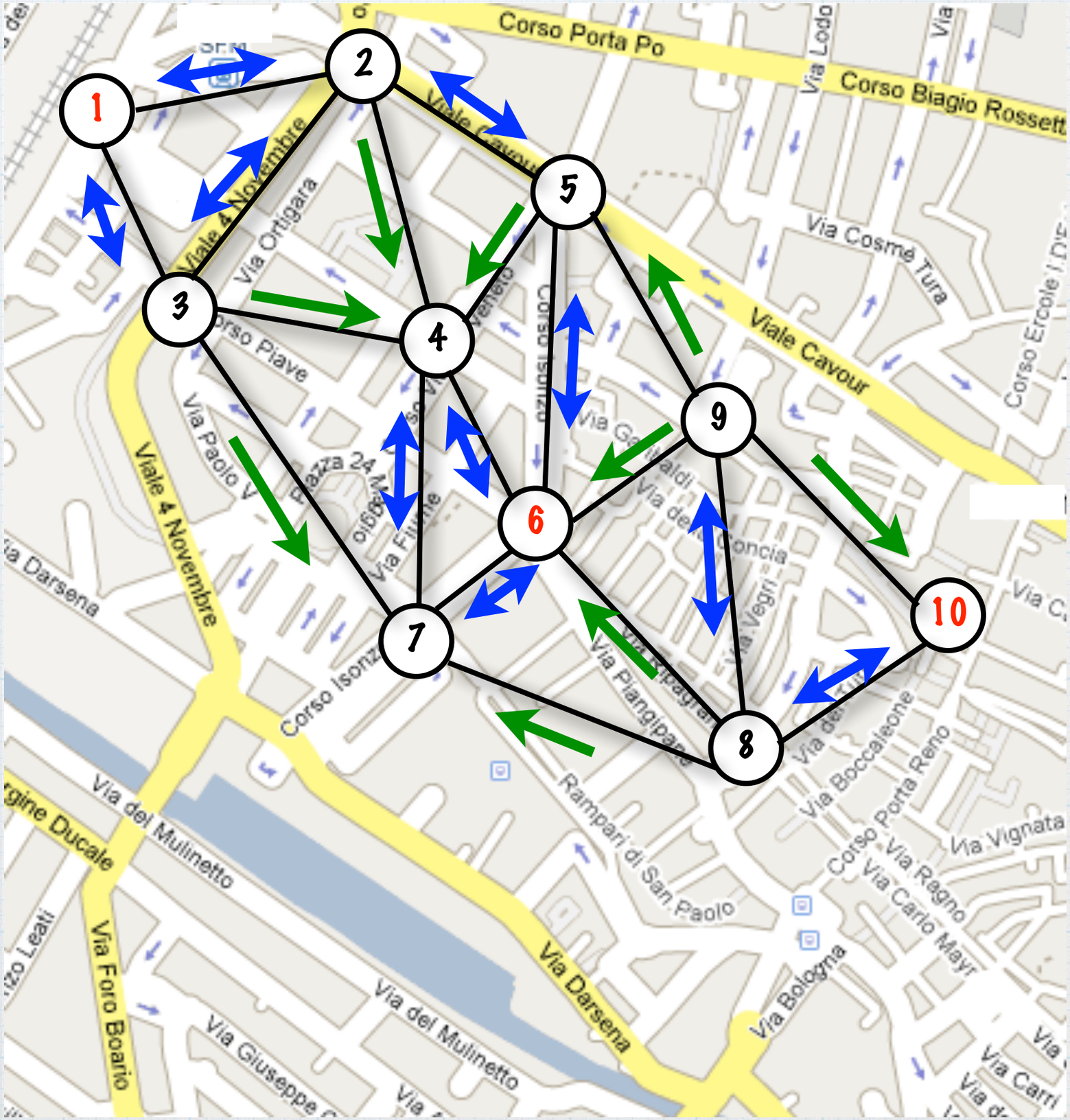
* Basic graph notation
  * graph representation
  * directed path
  * cut
* Remember the principles of modeling
* Basics of asymptotic computational complexity analysis

Traffic plan of the city

intersection

street

assign a direction to the arcs:
one way or
two ways streets



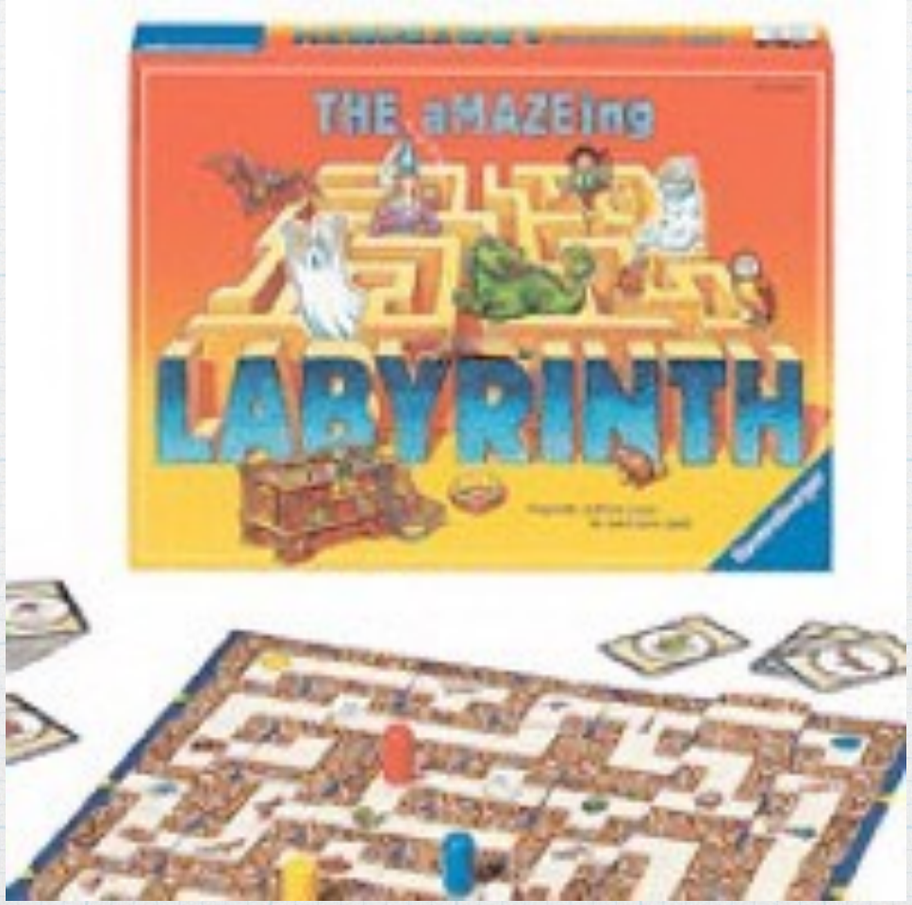
Connection problems

Is there a path from 1 to 6?

How can you certify it concisely?

Is there a path from 1 to 10?

How can you certify it concisely?



Graph_search (G, s, P)

```
for i ∈ N do P[i]←0
            P[s]←s; Q←{s};
repeat
    select i from Q; Q←Q\{i};
    foreach (i,j) ∈ FS(i) do
        if P[j]=0 then  P[j]← i
                        Q←Q∪{j}
until Q=∅
```

Very important property

Given a directed graph $G = (N, A)$

and two nodes $s, t \in N$

either there is a path p_{st}

or there is a cut $(N_s, N_t), s \in N_s, t \in N_t$

such that each arc of the cutset is $(i, j) : i \in N_t, j \in N_s$

[variant of the Minty coloring lemma]

Shortest path problem

Given a directed graph $G = (N, A)$
with costs $c_{ij}, \forall (i,j) \in A$
and two nodes $s, t \in N$

find the (elementary) path p from s to t
minimizing $\sum_{(i,j) \in p} c_{ij}$

Renewal plan problem

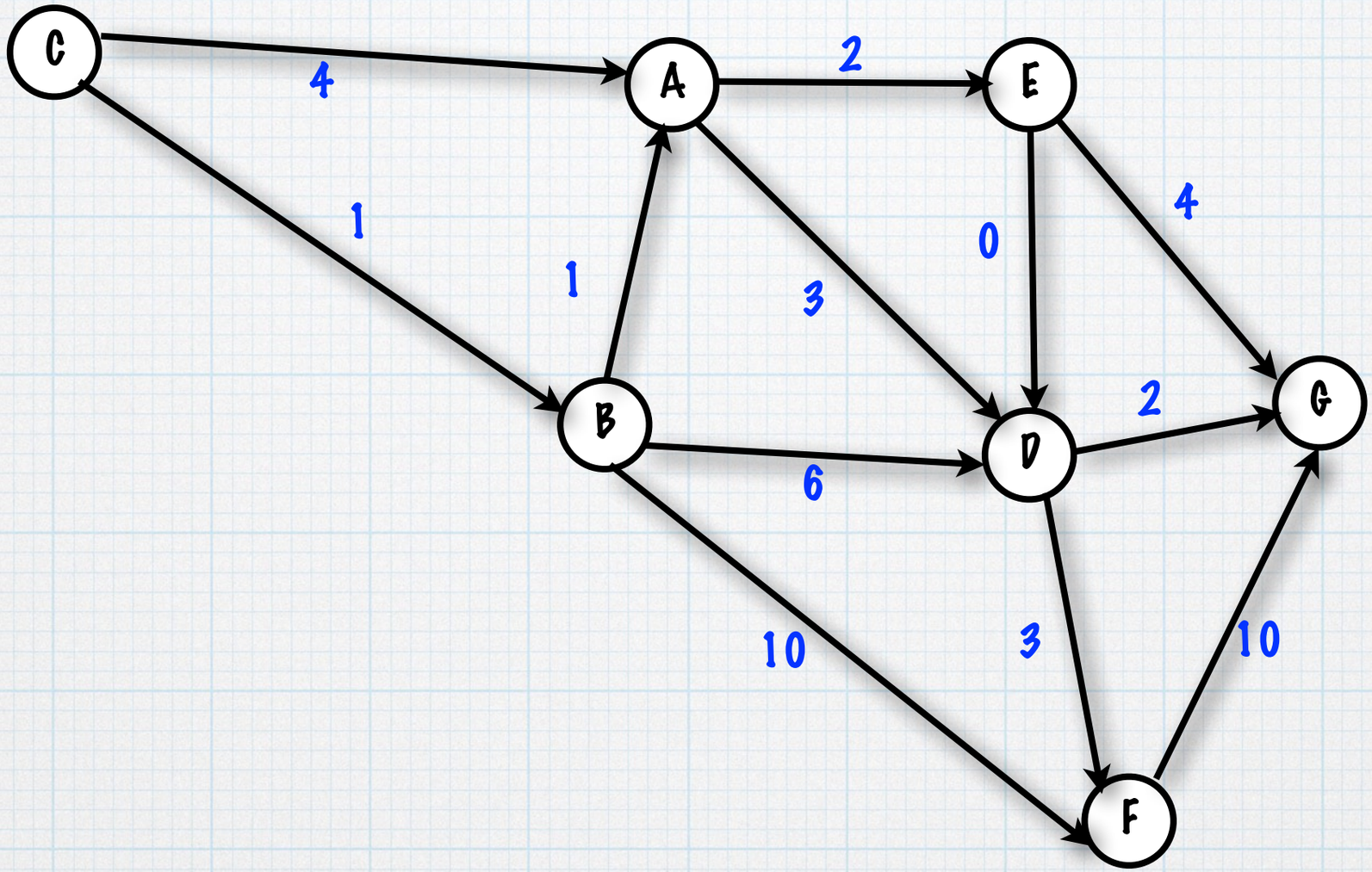
You are hired for a 5 years job, a car is required and you don't have one

	cost of the car
now	12
+1yr	13
+2yrs	14
+3yrs	15
+4yrs	16

age	mainte nance	selling price
1	2	7
2	4	6
3	5	3
4	9	2
5	11	1

When to buy and sell a car?

When G is acyclic

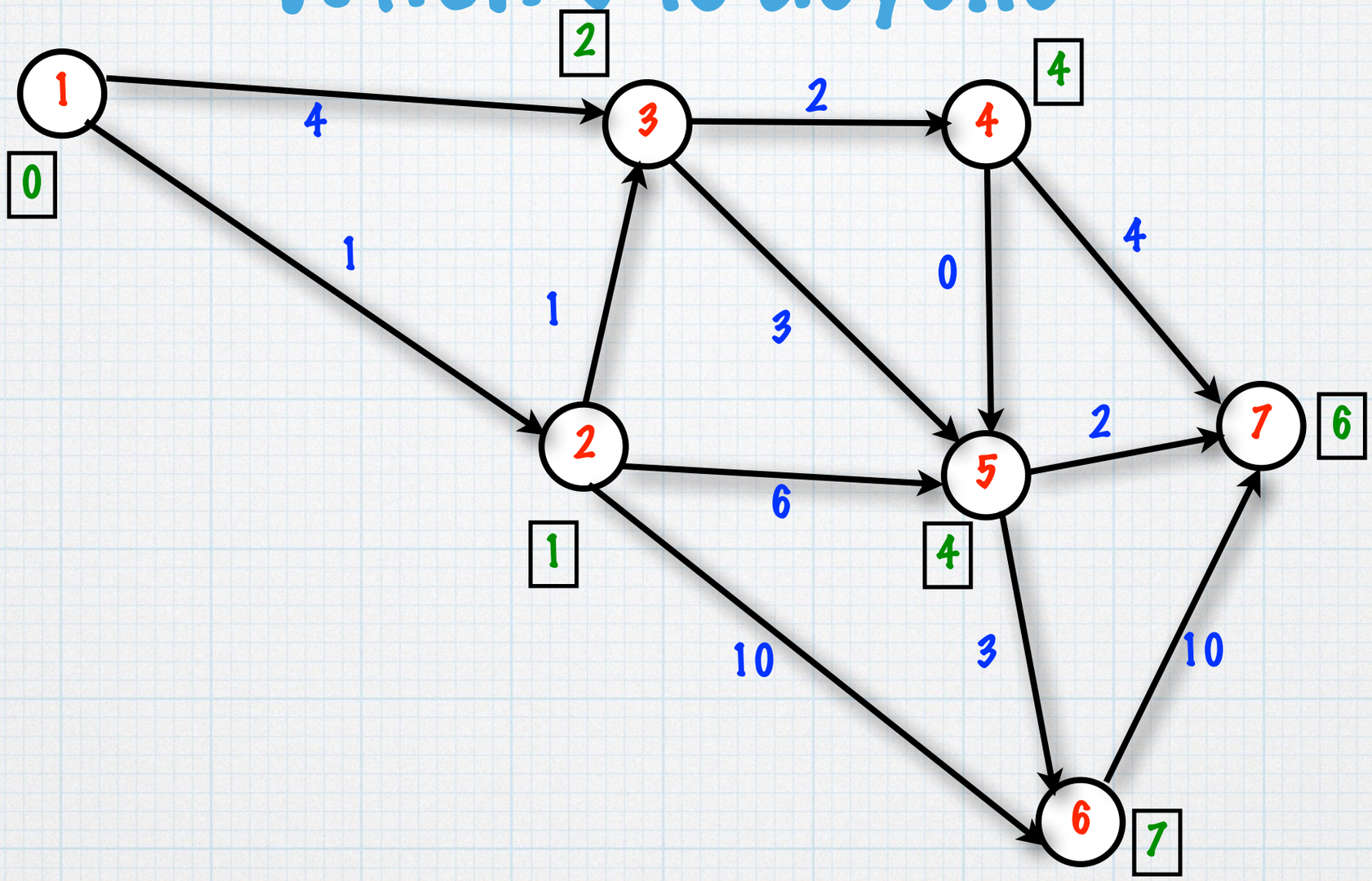


How can we check that there are no cycles?

Topological order

$$(i, j) \in A \Rightarrow i < j$$

When G is acyclic



Length of the shortest path from 1 to 1: $d[1] = 0$

Inductive rule

$$d[i] = \min\{d[j] + c_{ji} : (j, i) \in BS(i)\}$$

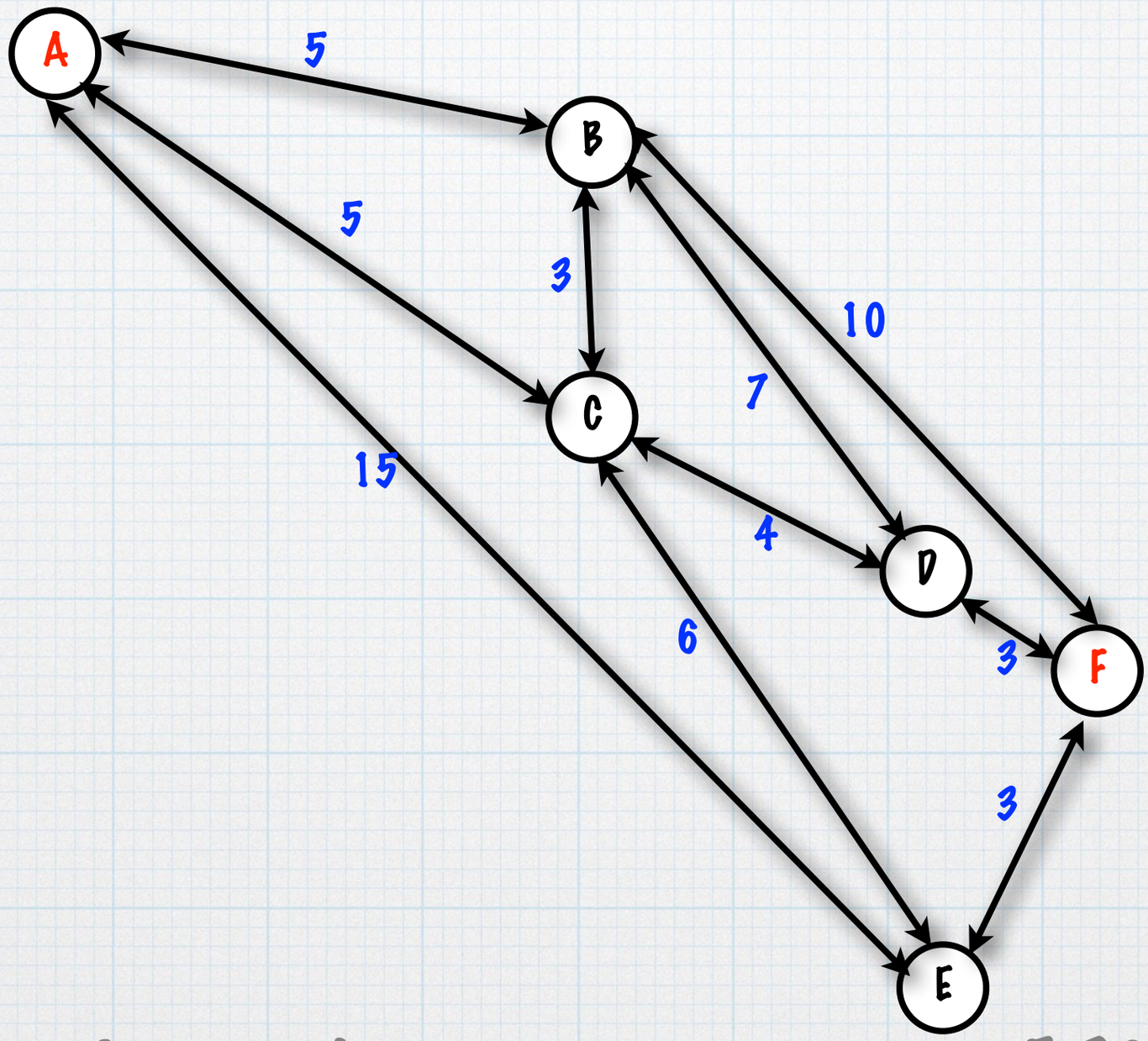
Note that $j < i$

SPT_Acyclic(G, P, d)

```
for i←2,...,n do P[i]←0; d[i]←+∞;

P[1]←1; d[1]←0;
for i←1,...,n-1 do
    foreach (i,j) ∈ FS(i) do
        if d[i]+c_ij < d[j] then  P[j]← i
                                  d[j] ← d[i]+c_ij
```

"Analog" computer



What is the relationship between π_i and d[i]?

Write your self reflections

Shortest Path Tree (SPT)

Given a directed graph $G = (N, A)$
with costs $c_{ij}, \forall (i,j) \in A$
and one node (**root**) $r \in N$

find the collection of all shortest paths from r

Why is it a tree?

Project planning: home construction

activity	description	time
A	foundations	7
B	doors and windows frames	2
C	floors	15
D	tubes	8
E	walls	10
F	ceiling	2
G	roof	5
H	doors and windows	8
I	wiring	2
J	paint	3

Precedences

$E \rightarrow D, G$

$A, B \rightarrow E$

$D, G \rightarrow F$

$G \rightarrow H$

$C, F \rightarrow I$

$I \rightarrow J$

Plan (sequence) the activities so that the completion time is minimized

LongestPT_Acyclic(G, P, d)

for i←2,...,n do P[i]←0; $\boxed{d[i] \leftarrow -\infty;}$

P[1]←1; d[1]←0;

for i←1,...,n-1 do

foreach (i,j) ∈ FS(i) do

if $\boxed{d[i]+c_{ij} > d[j]}$ then P[j]← i

$d[j] \leftarrow d[i]+c_{ij}$

Blended activity: dynamic programming

$$\max 7x_1 + 3x_2 + 3x_3 + 1x_4$$

$$3x_1 + 1x_2 + 3x_3 + 2x_4 \leq 6$$

$$x_i \in \{0,1\}, i = 1,\ldots,4$$

Read Section 5.6

Construct the DP graph

When G is not acyclic

Advantages of SPT_Acyclic algorithm:

nodes (and arcs) are **examined only once** according to the order

when a node i is considered **d[i] has the final value**

How can we maintain this behavior if G is no more acyclic?

Hypothesis:

$$c_{ij} \geq 0, \forall (i,j) \in A$$

SPT_Dijkstra(G, r, P, d)

for $i \in N$ do $P[i] \leftarrow 0$ $d[i] \leftarrow +\infty$;

$P[r] \leftarrow r$; $d[r] \leftarrow 0$; $Q \leftarrow \{r\}$;

repeat

select i from Q with minimum d[i]; $Q \leftarrow Q \setminus \{i\}$;

foreach $(i,j) \in FS(i)$ do

if $d[i]+c_{ij} < d[j]$ then $P[j] \leftarrow i$

$d[j] \leftarrow d[i]+c_{ij}$

$Q \leftarrow Q \cup \{j\}$

until $Q=\emptyset$

Dijkstra (1930-2002)

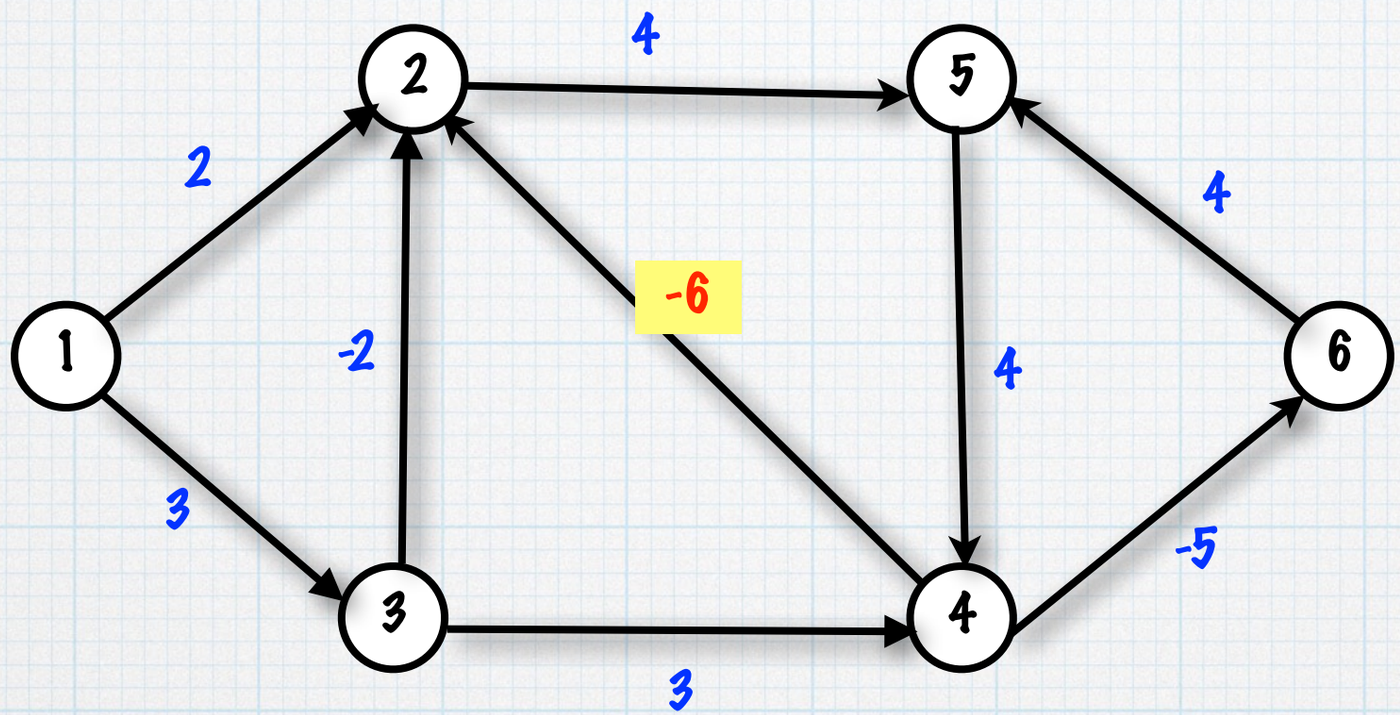
* applet for generating and solving problems

  https://www-m9.ma.tum.de/graph-algorithms/spp-dijkstra/index_en.html

* Dijkstra home page with his original slides

  http://www.cs.utexas.edu/users/EWD/

Graphs with cycles and negative costs

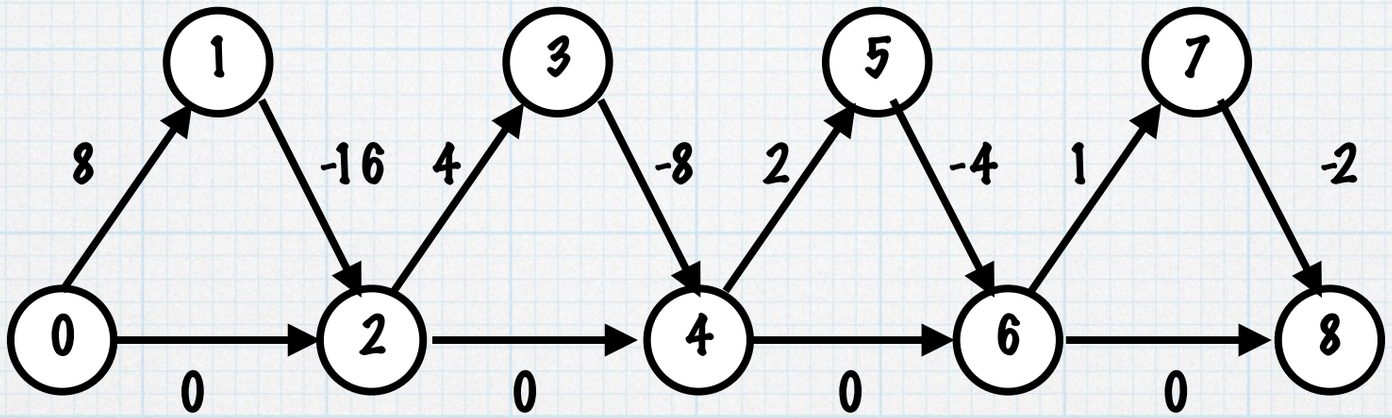


Warning

What happens to Dijkstra algorithm with negative cost cycles?

What happens if we apply it to a graph without negative cost cycles but with some negative cost arcs?

Dijkstra algorithm killer



How many times each node is introduced into Q?

What are the values of d[8] during the execution?

SPT_Label_Correcting(G, r, P, d)

for $i \in N$ do $P[i] \leftarrow 0$ $d[i] \leftarrow +\infty$;

$P[r] \leftarrow r$; $d[r] \leftarrow 0$; $Q \leftarrow \{r\}$;

repeat

select i from Q with FIFO policy; $Q \leftarrow Q \setminus \{i\}$;

foreach $(i,j) \in FS(i)$ do

if $d[i]+c_{ij} < d[j]$ then $P[j] \leftarrow i$

$d[j] \leftarrow d[i]+c_{ij}$

$Q \leftarrow Q \cup \{j\}$

until $Q=\emptyset$

How to detect negative cost cycles

From the complexity analysis we know that a node cannot be inserted into Q more than **n-1 times**

If a node is inserted **n times**

negative cost cycle

How do we identify it?

ILO

* Apply SPT_Acyclic for shortest and longest paths
* Apply SPT_Dijkstra
* Apply SPT_L_queue
* Recall the different worst case complexity of the SPT algorithm
* Understand when to apply the most efficient algorithm

Chapter 2, section 4 and 5